

머신러닝

로지스틱 회귀 기초

Chapter 8



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 로지스틱 회귀란?

02 분류 문제의 성능지표

03 사이킷런을 이용한 로지스틱 회귀

01 로지스틱 회귀란?



분류 문제

- 분류 모델: 몇 가지 이산적 값(일반적으로 범주형 데이터) 중 하나를 선택하는 모델
 - GRE와 GPA 데이터를 통해 대학의 합격 여부(Admit 열)를 나타냄
 - GRE와 GPA 정보를 산점으로 표현
 - 합격자는 파란색, 불합격자는 빨간색으로 나타냄

표 9-1 GRE와 GPA 정보를 활용하여 합격 여부를 나타내는 데이터

Number	Admit	GRE	GPA	Number	Admit	GRE	GPA
1	0	380	3.61	16	0	660	3.34
2	1	660	3.67	17	1	740	4.00
3	1	800	4.00	18	0	560	3.19
4	1	640	3.19	19	0	380	2.94
5	0	520	2.93	20	0	400	3.65
6	1	760	3.00	21	0	600	2.82
7	1	560	2.98	22	1	620	3.18
8	0	400	3.08	23	0	560	3.32
9	1	540	3.39	24	0	640	3.67
10	0	700	3.92	25	1	680	3.85
11	0	800	4.00	26	0	580	4.00
12	0	440	3.22	27	0	600	3.59
13	1	760	4.00	28	0	740	3.62
14	0	700	3.08	29	0	620	3.30
15	1	700	4.00	30	0	580	3.69

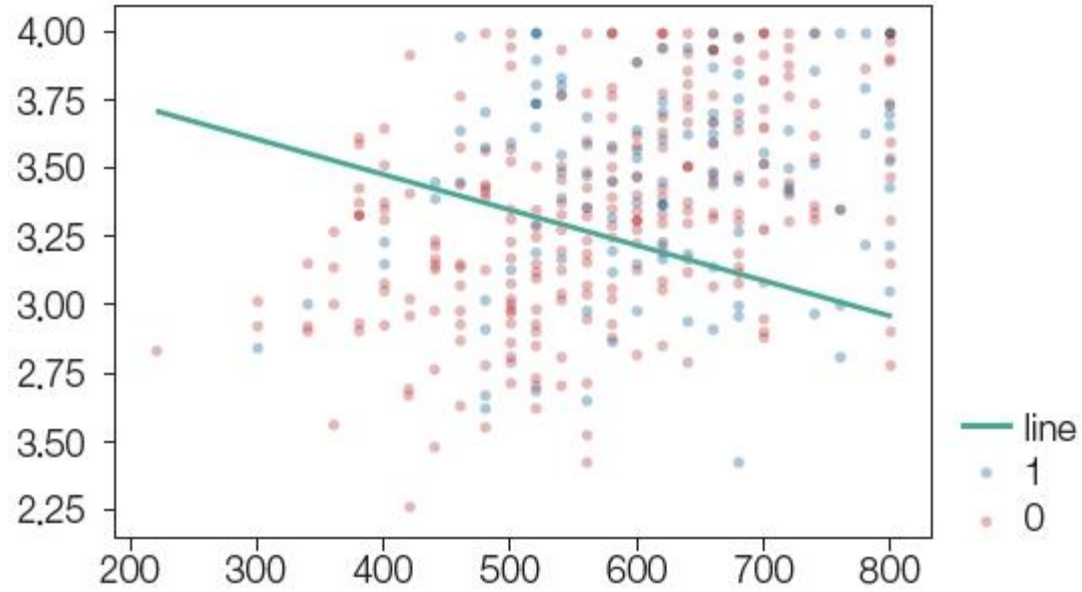


그림 9-1 산점도 위에 선형회귀 모델 표현



분류 문제

- 초록색 선을 추가해 선 상단은 합격, 선 하단은 불합격

- 아래 수식으로 기존 선형회귀 모델을 적용

$$f(x) = 4 - 0.0013 \times GRE - GPA$$

$$Admit = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- 문제점

- $f(x)$ 의 값이 1 이상이나 0 이하로 나올 수 있는데 각 피쳐들이 Y에 영향을 주는 것을 해석하는 문제
- 사건의 발생 여부는 이산적인데 실제 $f(x)$ 수식은 연속적

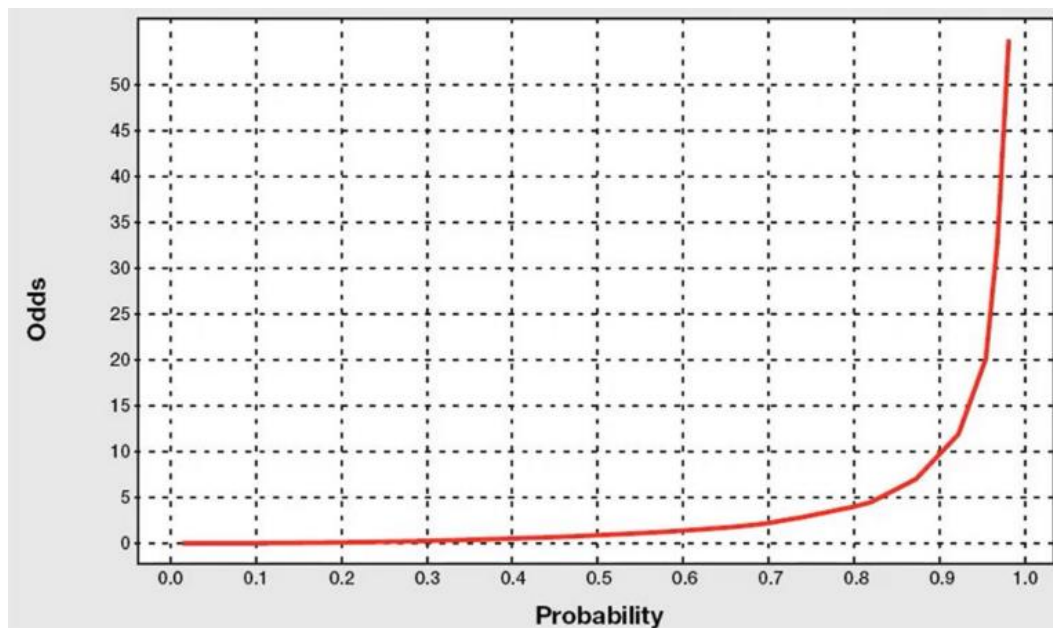


해결방안: 오즈

- 이진 분류(binary classification) 문제를 확률로 표현
- 오즈(Odds): 어떤 사건이 일어날 확률을 $P(X)$ 로 나타내고 일어나지 않을 확률을 $1 - P(x)$ 로 나타냄

$$\frac{P(X)}{1 - P(X)} \quad (0 \leq P(X) \leq 1)$$

- 확률이 올라갈수록 오즈도 급속히 상승

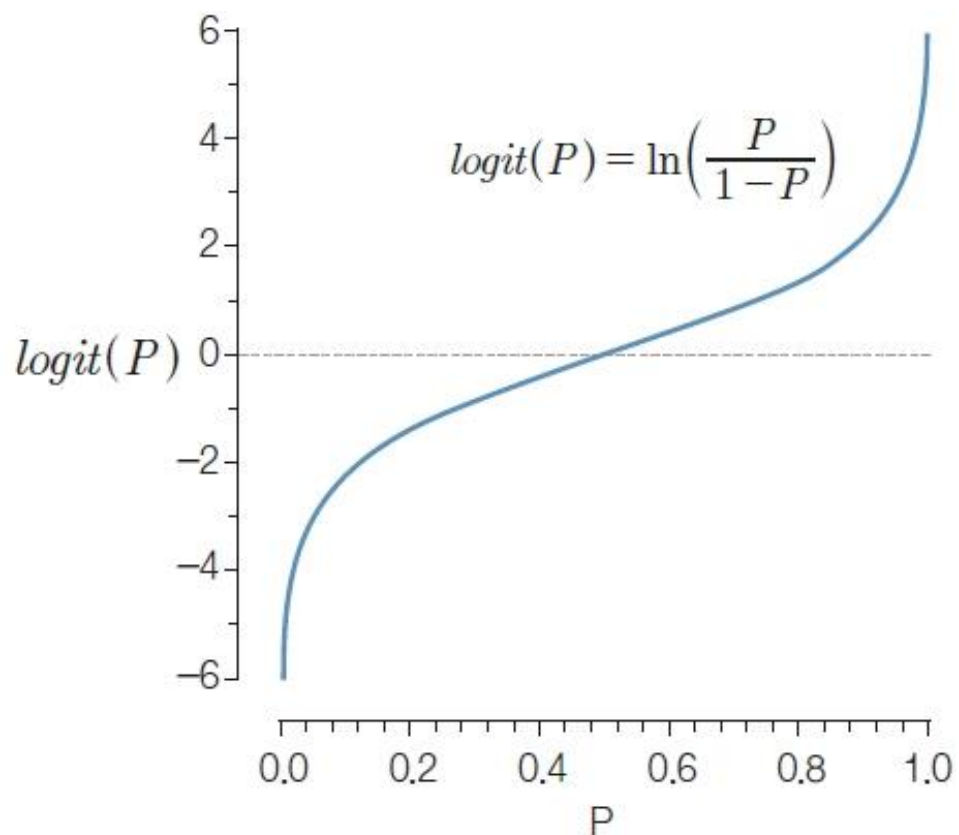




로짓(logit) 함수

: 오즈에 상용로그를 붙인 수식

- Why? 음의 무한대까지 포함시키기 위해



$$\begin{aligned}logit(p(y = 1 | x)) &= \log_e\left(\frac{p}{1-p}\right) \\&= \log_e(p) - \log_e(1-p) \\&= -\log_e\left(\frac{1}{p} - 1\right)\end{aligned}$$

로그의 특징

- $\log_a 1 = 0, \log_a a = 1$
- $\log_a MN = \log_a M + \log_a N$
- $\log_a \frac{M}{N} = \log_a M - \log_a N$
- $\log_a L^k = k \log_a L$

그림 9-3 로짓 함수 그래프



로지스틱 함수(logistic function)

- 로지스틱 함수(logistic function) : 로짓 함수의 역함수
 - X 값으로 확률을 넣으면 $\text{logit}(P)$ 꼴로 나타남
 - 확률을 구하려면 기존 함수의 역함수를 취하여 연산
 - 그래프가 S자 커브 형태인 시그모이드 함수(sigmoid function)

e: 자연상수 (Euler's number)

$e \approx 2.718281828$

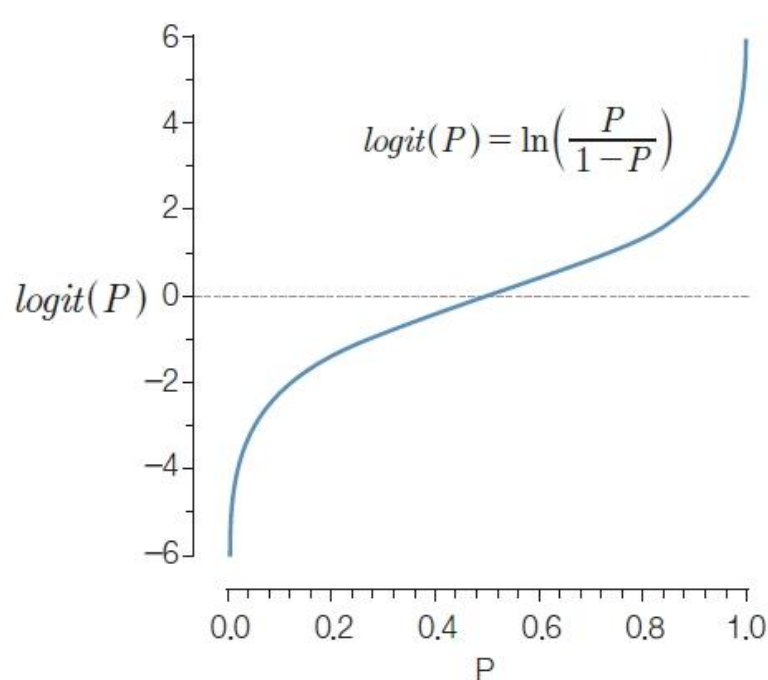


그림 9-3 로짓 함수 그래프

$$f(x) = y = -\log_e\left(\frac{1}{x} - 1\right)$$

$$x = -\log_e\left(\frac{1}{y} - 1\right)$$

$$e^{-x} = \frac{1-y}{y}$$

$$y \times e^{-x} + y = 1$$

$$y(e^{-x} + 1) = 1$$

$$y(e^{-x} + 1) = 1$$

$$y = \frac{1}{1 + e^{-x}}$$

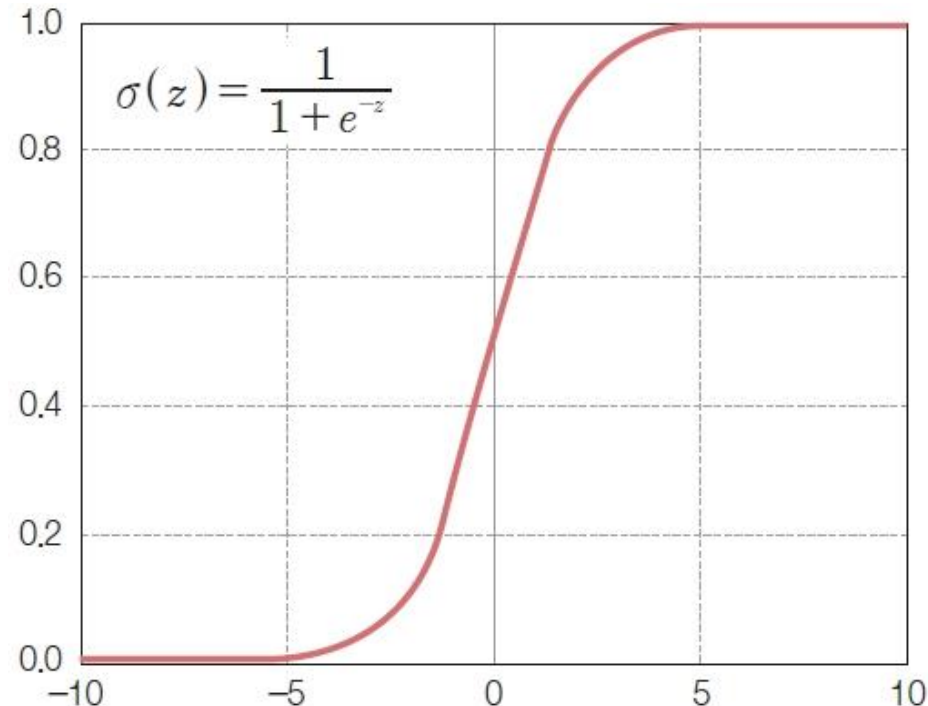


그림 9-4 시그모이드 함수



로지스틱 회귀(Logistic Regression)

: 종속변수가 이진 값(0 또는 1) 일 때 사용하는 분류 모델

- 가설함수(hypothesis function)

$$h_{\theta}(x) = g(z) = \frac{1}{1 + e^{-z}} \quad z = w_0x_0 + w_1x_1 + \dots + w_nx_n = \theta^T X$$

- z 는 가중치 값과 피쳐 값의 선형 결합
- 가중치 값을 찾는 학습을 위해 경사하강법 알고리즘 사용

- 비용함수(cost function)

- 먼저 비용함수를 정의하고 예측값과 실제값 간의 차이를 최소화하는 방향으로 학습
- 실제값이 1일 때와 실제값이 0일 때 각각 다르게 비용함수를 정의

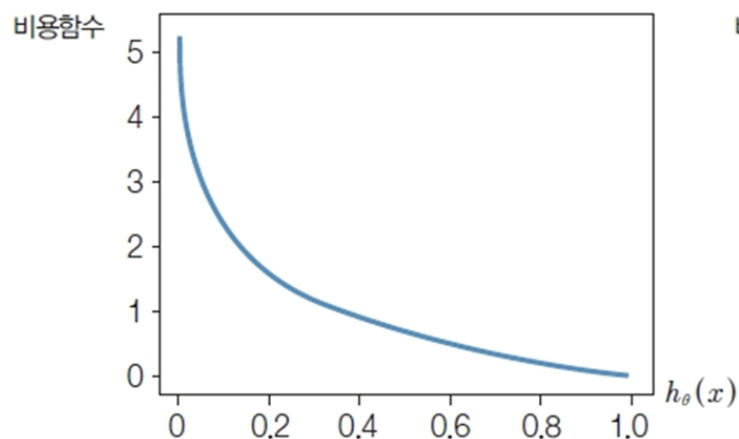
$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases} \approx 0$$

- $\log 1 = 0$
- "-" 는 항상 양수를 나오게 하기 위함

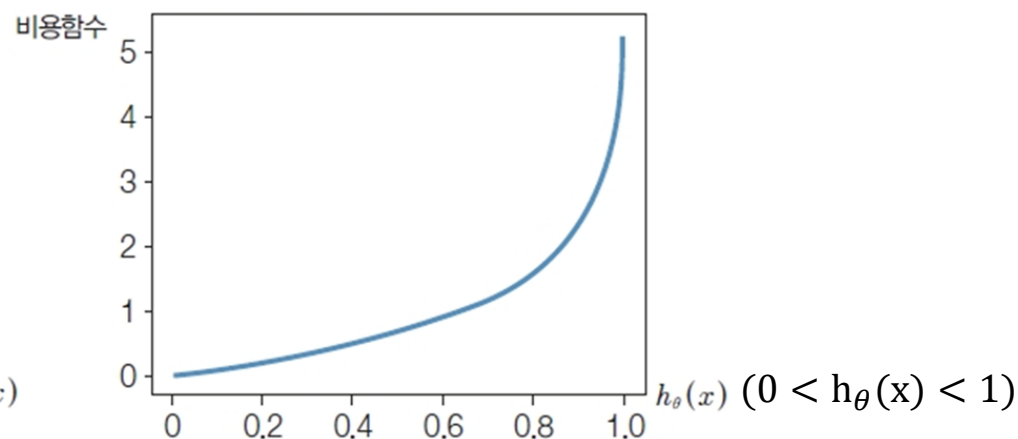


비용함수

- (a)는 $y = 1$ 일 때, (b)는 $y = 0$ 일 때 비용함수 그래프



(a) $-\log(h_\theta(x))$



(b) $-\log(1-h_\theta(x))$

- (a)에서 h 값이 1에 가까워질수록 비용함수가 0에 가까워짐
- (b)에서 h 값이 0에 가까워질수록 비용함수가 0에 가까워짐

- 두 경우의 비용함수를 하나로 통합

$y = 0$ 일 경우, 함수 생존

$y = 1$ 일 경우, 함수 생존

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m} \sum_{i=1}^m [\underbrace{y^{(i)} \log h_\theta(x^{(i)})}_{\text{blue}} + \underbrace{(1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))}_{\text{red}}]$$

02

분류 문제의 성능지표



혼동행렬

- 혼동행렬(confusion matrix): 예측값이 실제값 대비 얼마나 잘 맞는지 2×2 행렬로 표현
 - True Positive (TP) : 실제값과 예측값이 모두 1인 경우
 - True Negative (TN) : 실제값과 예측값이 모두 0인 경우
 - False Negative (FN) : 실제값은 1이지만 예측값이 0인 경우
 - False Positive (FP) : 실제값은 0이지만 예측값이 1인 경우

		예측값 (Predicted)	
		0	1
실제값 (Actual)	0	True Negative (TN)	False Positive (FP)
	1	False Negative (FN)	True Positive (TP)



혼동행렬 코드

```
from sklearn.metrics import confusion_matrix
```

```
y_true = [1, 0, 1, 1, 0, 1]
```

```
y_pred = [0, 0, 1, 1, 0, 1]
```

```
confusion_matrix(y_true, y_pred) #실제값 행 번호, 예측값 열 번호
```

```
array([[2, 0],  
       [1, 3]], dtype=int64)
```

		예측값 (Predicted)	
		0	1
실제값 (Actual)	0	True Negative (TN)	False Positive (FP)
	1	False Negative (FN)	True Positive (TP)

```
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).flatten()  
(tn, fp, fn, tp)
```

```
(2, 0, 1, 3)
```



정확도

- 정확도(accuracy) : 전체 데이터 개수 대비 정답을 맞춘 데이터의 개수
 - 이때 맞힌 정답이 긍정(Positive)이든 부정(Negative)이든 상관없음

```
import numpy as np
```

```
y_pred = np.array([0, 1, 1, 0])
```

```
y_true = np.array([0, 1, 0, 0])
```

```
sum(y_true == y_pred) / len(y_true)
```

0.75

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y_true, y_pred)
```

0.75

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

		예측값 (Predicted)	
		0	1
실제값 (Actual)	0	True Negative (TN)	False Positive (FP)
	1	False Negative (FN)	True Positive (TP)



재현율(recall)

- 실제 Positive인 데이터 중에서 모델이 Positive로 맞게 예측한 비율

$$RECALL(TPR) = \frac{TP}{TP + FN}$$

- 실제 정답 중 얼마나 많이 찾아냈는지 평가하는 지표
- FN (놓친 것)을 줄일수록 재현율이 올라감
 - Positive 데이터를 놓치지 않는 능력을 의미함
 - 예)의료 진단, 이상 탐지, 보안 분야에서 중요

		예측값 (Predicted)	
		0	1
실제값 (Actual)	0	True Negative (TN)	False Positive (FP)
	1	False Negative (FN)	True Positive (TP)

```
from sklearn.metrics import recall_score
```

```
recall_score(y_true, y_pred)
```

```
0.8333333333333333
```



정밀도(precision)

- 모델이 Positive라고 예측한 것 중 실제로 Positive인 비율

$$PRECISION(PPV) = \frac{TP}{TP + FP}$$

- 모델이 Positive라고 판단한 결과가 얼마나 정확한지 평가하는 지표
- 잘못된 Positive 예측(FP)을 얼마나 줄였는지를 나타냄
 - FP를 줄일수록 정밀도 증가
 - 예)스팸 필터, 추천 시스템 등에서 중요

		예측값 (Predicted)	
		0	1
실제값 (Actual)	0	True Negative (TN)	Faise Positive (FP)
	1	False Negative (FN)	True Positive (TP)

```
from sklearn.metrics import precision_score
```

```
precision_score(y_true, y_pred)
```

0.75



정밀도 vs 재현율



Precision (정밀도)	Recall (재현율)
모델이 양성이라고 예측한 것 중 실제로 양성인 비율	실제 양성 데이터 중 모델이 찾아낸 비율
잘못된 양성 예측(False Positive)을 줄이는 것이 중요	놓치는 양성(False Negative)을 줄이는 것이 중요
예측의 정확성에 초점	탐지 능력에 초점
예: 스팸 메일 분류	예: 질병 진단



F1-스코어(F1-Score)

- 일반적으로 정밀도와 재현율은 트레이드오프(trade-off) 관계
- 정밀도가 높으면 재현율이 낮고, 재현율이 높으면 정밀도가 낮음
- 이러한 트레이드오프 문제를 해결하기 위해 정밀도와 재현율의 조화 평균을 이용한 것이 F1-스코어 평가
 - 정밀도와 재현율이 서로 균형을 이루는지를 확인할 때 사용

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall}$$

- 두 지표가 모두 높을 때 F1 Score도 높아짐
- 불균형 데이터(Class Imbalance)에서 모델 성능 평가에 유용

```
from sklearn.metrics import f1_score
```

```
f1_score(y_true, y_pred)
```

```
0.7333333333333334
```

03

사이킷런을 이용한 로지스틱 회귀



데이터 설명

- 인터넷 사용자의 인구통계 정보, 경제 수준, 직업, 인터넷 사용 기간 등을 포함한 설문 데이터
- 목표: 사용자의 특성을 보고 신규 인터넷 사용자인지 예측

컬럼	설명	예시
no	인덱스	1
who	사용자 ID	kdr4904
Newbie	인터넷 신규 사용자 여부 - y (1=신규, 0=기존 사용자)	0
Age	나이	54
Gender	성별	Male
Household Income	가구 소득 수준	\$30-74
Sexual Preference	성적 성향	Heterosexual
Country	거주 지역	Ontario
Education Attainment	최종 학력	Some College
Major Occupation	직업	Computer
Marital Status	결혼 상태	Married
Years on Internet	인터넷 사용 기간	1-3 yr



데이터 준비

```
import pandas as pd

df = pd.read_csv("./data/UVA_Internet_Survey_Dataset.csv")
df.head()
```

	no	who	Newbie	Age	Gender	Household Income	Sexual Preference	Country	Education Attainment	Major Occupation	Marital Status	Years on Internet
0	0	id74364	0	54.0	Male	\$50-74	Gay male	Ontario	Some College	Computer	Other	4-6 yr
1	1	id84505	0	39.0	Female	Over \$100	Heterosexual	Sweden	Professional	Other	Other	1-3 yr
2	2	id84509	1	49.0	Female	\$40-49	Heterosexual	Washington	Some College	Management	Other	Under 6 mo
3	3	id87028	1	22.0	Female	\$40-49	Heterosexual	Florida	Some College	Computer	Married	6-12 mo
4	4	id76087	0	20.0	Male	\$30-39	Bisexual	New Jersey	Some College	Education	Single	1-3 yr



데이터 전처리: 드롭

```
df.pop('no') #no열 제거  
df.pop('who') #who열 제거  
df.pop('Country') #Country열 제거  
df.pop('Years on Internet') #Years on Internet열 제거
```

```
df.dtypes
```

```
Newbie                int64  
Age                   float64  
Gender                object  
Household Income      object  
Sexual Preference      object  
Education Attainment  object  
Major Occupation      object  
Marital Status        object  
dtype: object
```



데이터 전처리: 데이터 타입 변환

- 데이터 타입 변환: 범주형 데이터는 "category" 타입으로 변환

```
category_cols = ['Gender', 'Household Income',  
                 'Sexual Preference', 'Education Attainment',  
                 'Major Occupation', 'Marital Status']
```

```
for col in category_cols:  
    df[col] = df[col].astype('category') # 범주형 데이터 타입
```

```
df.dtypes
```

Newbie	category
Age	float64
Gender	category
Household Income	category
Sexual Preference	category
Education Attainment	category
Major Occupation	category
Marital Status	category
dtype:	object

```
df["Gender"]
```

0	Male
1	Female
2	Female
3	Female
4	Male
	...
19578	Male
19579	Male
19580	Female
19581	Female
19582	Female

```
Name: Gender, Length: 19583, dtype: category  
Categories (2, object): ['Female', 'Male'] _3
```

▶ 데이터 전처리: 원 핫 인코딩 + 결측값 처리

- 원핫인코딩 `df_onehot = pd.get_dummies(df) #category타입만 변환`
`df_onehot.shape`

(19583, 38)

- 결측값 확인하여 채우기

```
df_onehot.isnull().sum()
```

```
Newbie          0
Age             561
Gender_Female    0
Gender_Male      0
```

(이하 생략)

```
df_onehot.fillna({"Age": df_onehot['Age'].mean()}, inplace=True)
df_onehot.isnull().sum()
```

```
Newbie          0
Age             0
Gender_Female    0
Gender_Male      0
```

(이하 생략)

▶ 데이터 전처리: x와 y 데이터 추출 및 정규화

- 독립변수 x와 종속변수 y 추출

```
y_data = df_onehot.pop("Newbie").values
x_data = df_onehot.values

y_data.shape, x_data.shape
((19583, 1), (19583, 37))
```

- "StandardScaler"를 사용하여 z-score 정규화

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

#x_scaled = scaler.fit(X)          # 평균과 표준편차 계산
#x_scaled = scaler.transform(X)    # 데이터 변환
x_scaled = scaler.fit_transform(x_data)
x_scaled[0]
```

```
array([ 1.50708807, -0.67494611,  0.67494611, -0.25651413, -0.3129664 ,
       -0.3647996 , -0.36336036,  1.99414645, -0.33613486, -0.33396655,
       ...])
(이하 생략)
```



데이터 전처리: 데이터 분리

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    x_scaled, y_data, test_size=0.33, random_state=42)

X_train.shape, X_test.shape

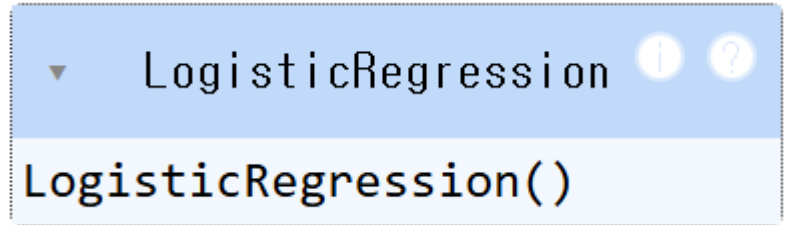
((13120, 37), (6463, 37))
```



로지스틱 회귀 학습

■ 사이킷런 LogisticRegression 객체 사용

```
from sklearn.linear_model import LogisticRegression  
  
logreg = LogisticRegression(fit_intercept=True)  
logreg.fit(X_train, y_train.flatten())
```



매개변수	의미	주요 값	설명
penalty	규제 방식	'l1', 'l2', 'elasticnet', None	모델의 과적합을 방지하기 위한 규제 방법
solver	최적화 알고리즘	'lbfgs', 'sag' 등	가중치를 찾는 학습 알고리즘
max_iter	최대 반복 횟수	정수	모델 학습 반복 횟수
fit_intercept	절편 사용 여부	True, False	절편(bias)을 사용할지 여부
random_state	랜덤 시드	정수	결과 재현을 위한 난수 설정
multi_class	다중 분류 방식	'ovr', 'multinomial'	다중 클래스 분류 방법



로지스틱 회귀 모델로 예측

- 데이터에서 학습한 scaler를 이용하여 새로운 데이터인 테스트 데이터를 동일한 기준으로 변환
- 예측 확률 보기

```
re_x= scaler.transform(X_test)  # 데이터 변환  
  
#넘파이 소수점 표현 세팅(생략가능)  
np.set_printoptions(precision=4, suppress=True)  
  
logreg.predict_proba(re_x)
```



```
array([[0.2898, 0.7102],  
       [0.9998, 0.0002],  
       [0.8256, 0.1744],  
       ...,  
       [0.8221, 0.1779],  
       [0.991 , 0.009 ],  
       [0.7971, 0.2029]])
```

- 예측 확률을 기반으로 최종 분류 결과를 생성

```
logreg.predict(re_x)
```

```
array([1, 0, 0, ..., 0, 0, 0], dtype=int64)
```



성능평가

■ 혼동행렬 표

```
from sklearn.metrics import confusion_matrix

y_true = y_test.copy()
y_pred = logreg.predict(re_x)

confusion_matrix(y_true, y_pred)
```



```
array([[3807, 955],
       [ 929, 772]], dtype=int64)
```

■ 정확도, 정밀도, 재현율, F1 스코어

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

print("정확도:", accuracy_score(y_true, y_pred)) # 정확도
print("정밀도:", precision_score(y_true, y_pred)) # 정밀도
print("재현율:", recall_score(y_true, y_pred))    # 재현율
print("F1score:", f1_score(y_true, y_pred))      # F1 스코어
```



```
정확도: 0.7084945071948012
정밀도: 0.4470179502026636
재현율: 0.4538506760728983
F1score: 0.45040840140023336
```

- 전체 정확도는 70% 수준으로 괜찮아 보이지만
- 양성 클래스 예측 능력(Precision, Recall)이 약 45% 수준으로 낮아 양성 탐지 성능이 부족한 모델이라고 해석할 수 있음



SUMMARY

- 로지스틱 회귀: 이진 분류 문제에서 사용되는 모델
 - 오즈에 로그를 씌워 시그모이드 함수를 이용하여 입력값을 0~1 사이 확률로 변환하여 분류 수행
- 분류 문제에서 성능지표
 - 혼동행렬을 사용한 지표: TP, TN, FP, FN을 통해 분류 결과를 분석
 - 혼동행렬을 이용한 지표: 정확도, 정밀도, 재현율, F1-score 등
- 사이킷런을 이용한 로지스틱 회귀 구현을 위한 함수
 - 모델 생성: `LogisticRegression`
 - 학습: `fit()`
 - 예측: `predict()`, `predict_proba()`



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝